JavaScript

Generators, Proxy & WeakMap

Iterators · Infinite Sequences · Async Generators · Proxy Traps · WeakMap/WeakSet

```
TOPICS COVERED


GENERATORS -> function* yield yield* async generators

PROXY -> traps get set validation Reflect

WEAKMAP -> weak references private data memoization

WEAKSET -> unique object tracking GC-safe
```

PART 1 — GENERATORS

Generator Functions

Beginner

A regular function runs start to finish. A Generator function can **pause mid-execution** at each **yield** and resume later — returning an iterator object.

```
// Regular function — runs all at once, no pause

function regular() {

return 1;

return 2; // never reaches here

}


// Generator — pauses at each yield

function* generator() {

yield 1; // pause here, return 1

yield 2; // resume, pause here, return 2

yield 3; // resume, pause here, return 3

}


const gen = generator(); // doesn't run yet — returns iterator


console.log(gen.next()); // { value: 1, done: false }

console.log(gen.next()); // { value: 2, done: false }

console.log(gen.next()); // { value: 3, done: false }

console.log(gen.next()); // { value: undefined, done: true }
```

```
EXECUTION FLOW


generator() called

|

v

PAUSED at yield 1 -- gen.next() --> { value: 1, done: false }

|

v

PAUSED at yield 2 -- gen.next() --> { value: 2, done: false }

|
```

```
v

PAUSED at yield 3 -- gen.next() --> { value: 3, done: false }

|

v

DONE -- gen.next() --> { value: undefined, done: true }
```

1 Iterating Generators — for...of, Spread, Destructuring

```
function* colors() {

yield 'red';

yield 'green';

yield 'blue';

}


// for...of

for (const c of colors()) console.log(c); // red, green, blue


// Spread

const arr = [...colors()]; // ['red', 'green', 'blue']


// Destructuring

const [first, second] = colors(); // 'red', 'green'
```

2 Infinite Sequences — Safe Lazy Evaluation

Generators are perfect for infinite sequences — they only compute the next value when asked. No memory overflow, no blocking.

```
function* idGenerator() {

let id = 1;

while (true) { // infinite loop — safe because of yield!

yield id++;

}

}


const getId = idGenerator();

console.log(getId.next().value); // 1

console.log(getId.next().value); // 2

console.log(getId.next().value); // 3 ...forever, never crashes
```

3 Two-Way Communication with .next(value)

You can pass values back INTO a generator via **.next(value)**. The passed value becomes the result of the paused **yield** expression.

```
function* calculator() {

const x = yield 'Enter first number:'; // pauses, receives input

const y = yield 'Enter second number:'; // pauses, receives input

yield `Result: ${x + y}`;

}


const calc = calculator();

console.log(calc.next().value); // 'Enter first number:'

console.log(calc.next(10).value); // 'Enter second number:' (10 sent as x)

console.log(calc.next(20).value); // 'Result: 30' (20 sent as y)
```

4 yield* — Delegating to Another Generator

```
function* inner() { yield 'a'; yield 'b'; }


function* outer() {

yield 1;
```

```
yield* inner(); // delegate — forwards all inner values

yield 2;

}

console.log([...outer()]); // [1, 'a', 'b', 2]


// Real use — recursive flatten

function* flatten(arr) {

for (const item of arr) {

if (Array.isArray(item)) yield* flatten(item); // recurse

else yield item;

}

}

console.log([...flatten([1,[2,[3,4]],5])]); // [1,2,3,4,5]
```

5

Async Generators — Streaming Data

Async Generators

Expert

```
// Combine async/await + generators for streaming

async function* fetchPages(url) {

let page = 1;

while (true) {

const res = await fetch(`${url}?page=${page}`);

const data = await res.json();

if (!data.length) return; // stop when no more pages

yield data; // yield each page of results

page++;

}

}


// Consume with for await...of

async function processAll() {

for await (const page of fetchPages('/api/users')) {

console.log('Processing:', page); // next page fetched automatically
```

```
}

}
```

PART 2 — PROXY

Proxy — Intercept Object Operations

Intermediate -> Expert

A Proxy wraps an object and lets you intercept and customise fundamental operations like property access, assignment, deletion, and more.

```
const proxy = new Proxy(target, handler);

// | |

// | +-- object with TRAPS (interceptors)

// +----------- original object being wrapped


// Basic example — log all property access and writes

const user = { name: 'Akshay', age: 25 };


const proxy = new Proxy(user, {

get(target, prop) {

console.log(`Getting ${prop}`);

return target[prop];

},

set(target, prop, value) {

console.log(`Setting ${prop} = ${value}`);

target[prop] = value;

return true; // must return true on success!

}

});


proxy.name; // logs: Getting name -> 'Akshay'

proxy.age = 26; // logs: Setting age = 26
```

6 Proxy Use Case — Validation

```
function createValidatedUser(data) {

return new Proxy(data, {

set(target, prop, value) {

if (prop === 'age' && typeof value !== 'number')

throw new TypeError('age must be a number');
```

```
if (prop === 'name' && value.length < 2)

throw new Error('name too short');

target[prop] = value;

return true;

}

});

}


const user = createValidatedUser({ name: 'Akshay', age: 25 });

user.age = 'twenty'; // TypeError: age must be a number

user.name = 'A'; // Error: name too short

user.age = 26; // OK!
```

7 Proxy Use Cases — Defaults & Read-Only

```
// Default Values

function withDefaults(target, defaults) {

return new Proxy(target, {

get(obj, prop) {

return prop in obj ? obj[prop] : defaults[prop];

}

});

}


const config = withDefaults(

{ port: 8080 },

{ host: 'localhost', port: 3000, debug: false }

);

console.log(config.port); // 8080 (own value)

console.log(config.host); // 'localhost' (from defaults)


// Read-Only Object

function readOnly(target) {

return new Proxy(target, {
```

```
set() { throw new Error('Object is read-only!'); },

deleteProperty() { throw new Error('Cannot delete!'); },

});

}

const cfg = readOnly({ apiUrl: 'https://api.example.com' });

cfg.apiUrl = 'hacked'; // Error: Object is read-only!
```

8 All Proxy Traps — Reference

```
const handler = {

get(target, prop, receiver) {}, // obj.prop

set(target, prop, value, receiver) {}, // obj.prop = value

has(target, prop) {}, // prop in obj

deleteProperty(target, prop) {}, // delete obj.prop

apply(target, thisArg, args) {}, // fn() (functions only)

construct(target, args) {}, // new fn()

getPrototypeOf(target) {}, // Object.getPrototypeOf()

ownKeys(target) {}, // Object.keys()

defineProperty(t, prop, descriptor) {}, // Object.defineProperty()

isExtensible(target) {}, // Object.isExtensible()

};
```

9 Reflect — Always Use Inside Proxy Traps

Using target[prop] inside traps can break with prototype inheritance. Always use Reflect inside Proxy handlers for correct behaviour.

```
const proxy = new Proxy(user, {

get(target, prop, receiver) {

console.log(`Getting: ${prop}`);

return Reflect.get(target, prop, receiver); // CORRECT

// return target[prop]; can break with inheritance!

},

set(target, prop, value, receiver) {

console.log(`Setting: ${prop}`);
```

```
return Reflect.set(target, prop, value, receiver); // CORRECT

}

});
```

WeakMap — Leak-Safe Key-Value Store

Intermediate -> Expert

```
// Regular Map — strong reference, PREVENTS GC

const map = new Map();

let user = { name: 'Akshay' };

map.set(user, 'metadata');

user = null; // user STILL can't be GC'd — Map holds it!


// WeakMap — weak reference, does NOT prevent GC

const weakMap = new WeakMap();

let user2 = { name: 'Rahul' };

weakMap.set(user2, 'metadata');

user2 = null; // user2 CAN be GC'd — WeakMap won't hold it!
```

10 WeakMap — Private Class Data

```
const _private = new WeakMap();


class BankAccount {

constructor(balance) {

_private.set(this, { balance }); // truly private data

}

getBalance() {

return _private.get(this).balance;

}

deposit(amount) {

_private.get(this).balance += amount;

}

}


const acc = new BankAccount(1000);

console.log(acc.getBalance()); // 1000

console.log(acc.balance); // undefined — truly private!

// When acc is GC'd, _private entry also auto-removed
```

11 WeakMap — Memoization Without Leaks

```
const cache = new WeakMap();

function expensiveCalc(obj) {
if (cache.has(obj)) return cache.get(obj); // return cached
const result = obj.value * 2; // expensive computation
cache.set(obj, result);
return result;
}
// When obj is GC'd -> cache entry also GC'd automatically
// Regular Map would keep obj alive forever -> LEAK!
```

12 WeakSet — Unique Object Tracking

```
const processed = new WeakSet();

function processOnce(obj) {
if (processed.has(obj)) {
console.log('Already processed — skipping!');
return;
}
// ... do processing ...
processed.add(obj);
}

// Real use — DOM element tracking
const initialized = new WeakSet();
function initElement(el) {
if (initialized.has(el)) return; // skip already initialized
el.addEventListener('click', handler);
initialized.add(el);
}
// When el removed from DOM -> WeakSet entry auto-cleaned
```

COMPARISON TABLES

Generator vs Regular Function

Feature	Regular Function	Generator Function
Syntax	function fn()	function* fn()
Returns	Single value	Iterator object
Execution	Runs to completion	Can pause at yield
Resumable	No	Yes — via .next()
Infinite sequences	Dangerous (blocks)	Safe (lazy)
Memory large data	Loads all at once	One item at a time

WeakMap vs Map vs Object

Feature	Map	WeakMap
Key types	Any value	Objects only
Prevents GC	Yes (strong refs)	No (weak refs)
Iterable	Yes	No
Has .size	Yes	No
Best for	General key-value	DOM metadata, caching

Proxy vs Object.defineProperty

Feature	Object.defineProperty	Proxy
Intercepts	Single property	Entire object
Dynamic keys	Must define upfront	Catches any property
Array mutations	Cannot detect push/pop	Detects all
Delete trapping	No	Yes
Used by	Vue 2	Vue 3, modern libs

INTERVIEW Q & A

Top Interview Questions

Q: What is the difference between function and function*?

-> Regular functions run to completion in one go. Generator functions (function) can pause execution at each yield, returning an iterator that resumes step by step via .next().*

Q: What does yield* do?

-> Delegates to another iterable (generator, array, string). The outer generator forwards all values from the inner iterable before continuing its own execution.

Q: What is a Proxy and what are traps?

-> A Proxy wraps an object and intercepts fundamental operations through handler functions called traps — like get, set, has, deleteProperty. Every operation on the proxy goes through the corresponding trap.

Q: Why use Reflect inside Proxy traps?

-> Using Reflect ensures correct behaviour with prototype chains. target[prop] can break with inheritance, while Reflect.get(target, prop, receiver) preserves the correct this binding.

Q: What is the difference between WeakMap and Map?

-> WeakMap holds weak references to keys — if the key has no other references, GC can collect it and the entry is auto-cleaned. Map holds strong references preventing GC. WeakMap is also not iterable and has no .size.

Q: Name two practical use cases for Proxy.

-> 1) Validation — throw errors on invalid property assignments. 2) Default values — return fallback for missing properties. Also used for read-only objects, logging/debugging, and Vue 3's reactivity system.

Quick Reference

```
function* -> returns iterator. yield pauses. .next() resumes.

yield* -> delegate to inner generator/iterable.

Proxy -> new Proxy(target, handler) with traps for every op.

Reflect -> always use inside traps for correct inheritance.

WeakMap -> weak refs, not iterable, keys must be objects.

WeakSet -> unique object tracking, auto-cleans on GC.
```